Assignment- 1

Name: Kartik Pawar

PRN : 2124UCSM1045

Dept: Cyber Security

Lab : 1

Experiment 1: Implementation of Caesar Cipher

1. Aim

Write a program to perform encryption and decryption using Caesar Cipher.

2. Learning Objectives

Understand: Grasp the concept of substitution ciphers and modular arithmetic.

Apply: Develop a Python script suitable for Google Colab that correctly shifts alphabetic characters.

Analyze: Evaluate the vulnerability of shift ciphers to brute-force attacks in real-world scenarios.

3. Activity-Based Learning: Manual Calculation

Context: Secure Military Coordination (Historical to Modern). Julius Caesar historically used this cipher to communicate sensitive military strategies to his generals, preventing intercepted messages from being understood by enemies.

Plaintext Message: DROP AT SECTOR FIVE

Secret Shift Key Chosen (K): 5

Manual calculation of the first 3 encrypted letters (formula: new_position = (position + K) mod 26, where A = 0, B = 1, ... Z = 25):

Letter	Position (A=0)	+ K (5)	mod 26	New Letter
D	3	8	8	I
R	17	22	22	W
O	14	19	19	T

Result: D → I, R → W, O → T (i.e. "DRO..." encrypts to "IWT..."), confirming the algorithm's logic before coding.

Full encrypted message (shift 5, for reference): IWTU FY XHYTW KNAJ

4. Algorithm — Encryption Process

1. Prompt the user to input the plaintext string (P) and the integer shift key (K).

2. Initialize an empty string for the ciphertext.

3. Traverse each character C in the plaintext P:

   • If C is a letter, determine its ASCII offset (65 for Uppercase, 97 for Lowercase).

   • Calculate the new character position using: new_char = (ASCII_Value(C) - Offset + K) % 26 + Offset

   • Convert the new ASCII value back to a character and append it.

   • If C is not a letter, append it directly without modification.

4. Return the ciphertext.

5. Reference / Student's Colab Execution Code

```
def encrypt_caesar(text, shift):
    """
    Encrypts the given text using Caesar Cipher with the specified shift.
    """
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = 65 if char.isupper() else 97
            shifted_char = chr((ord(char) - ascii_offset + shift) % 26 + ascii_offset)
            result += shifted_char
        else:
            result += char
    return result


def decrypt_caesar(text, shift):
    """
    Decrypts the given text by shifting backwards.
    """
    return encrypt_caesar(text, -shift)

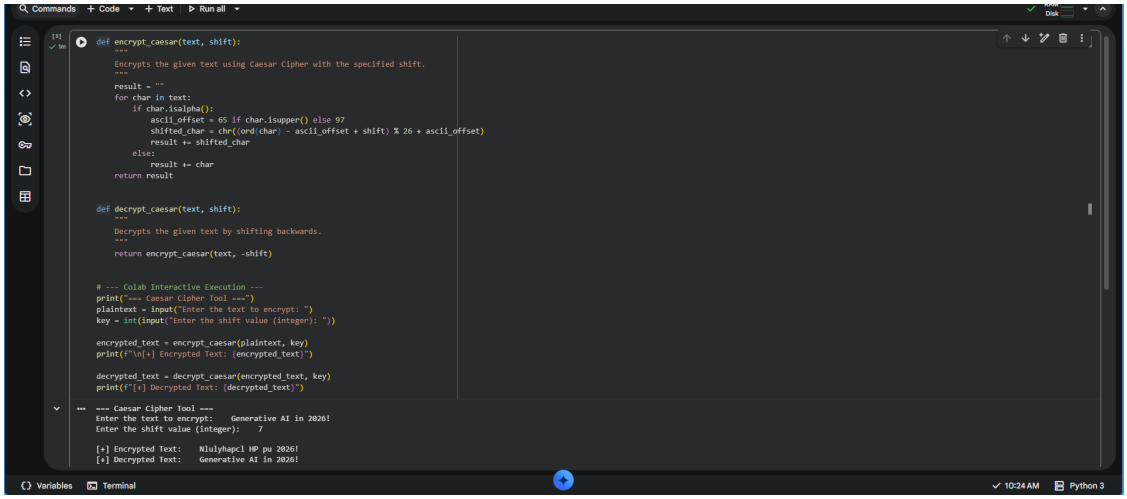

# --- Colab Interactive Execution ---
print("=== Caesar Cipher Tool ===")
plaintext = input("Enter the text to encrypt: ")
key = int(input("Enter the shift value (integer): "))

encrypted_text = encrypt_caesar(plaintext, key)
print(f"\n[+] Encrypted Text: {encrypted_text}")

decrypted_text = decrypt_caesar(encrypted_text, key)
print(f"[+] Decrypted Text: {decrypted_text}")
```

6. Sample Input and Output (Executed in Google Colab)

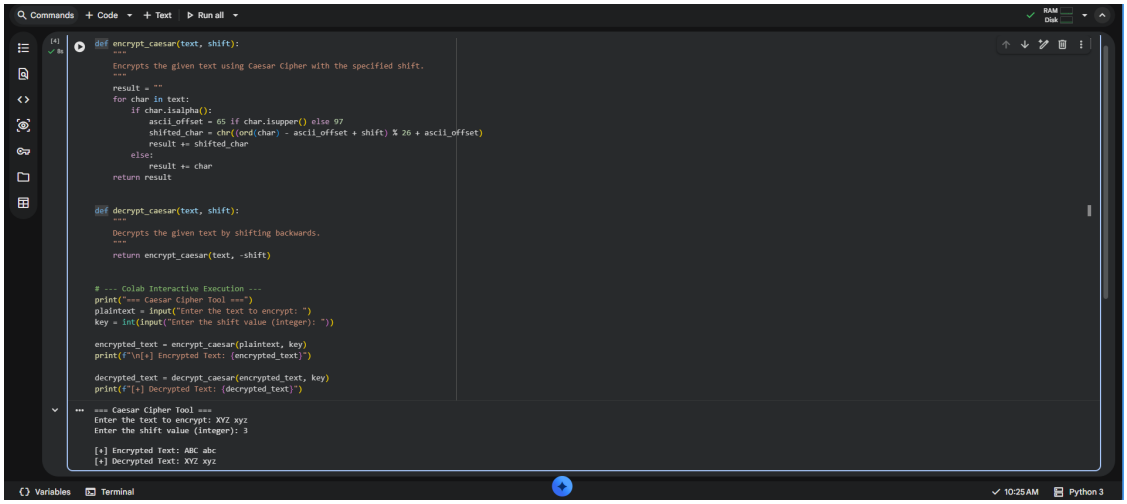
Execution — Test Case 1




```
Enter the text to encrypt:    Generative AI in 2026!
Enter the shift value (integer):    7

[+] Encrypted Text:    Nlulyhapcl HP pu 2026!
[+] Decrypted Text:    Generative AI in 2026!
```

Execution — Test Case 2 (Edge Case)




```
Enter the text to encrypt: XYZ xyz
Enter the shift value (integer): 3

[+] Encrypted Text: ABC abc
[+] Decrypted Text: XYZ xyz
```

7. Student's Observation (Custom Input & Output)

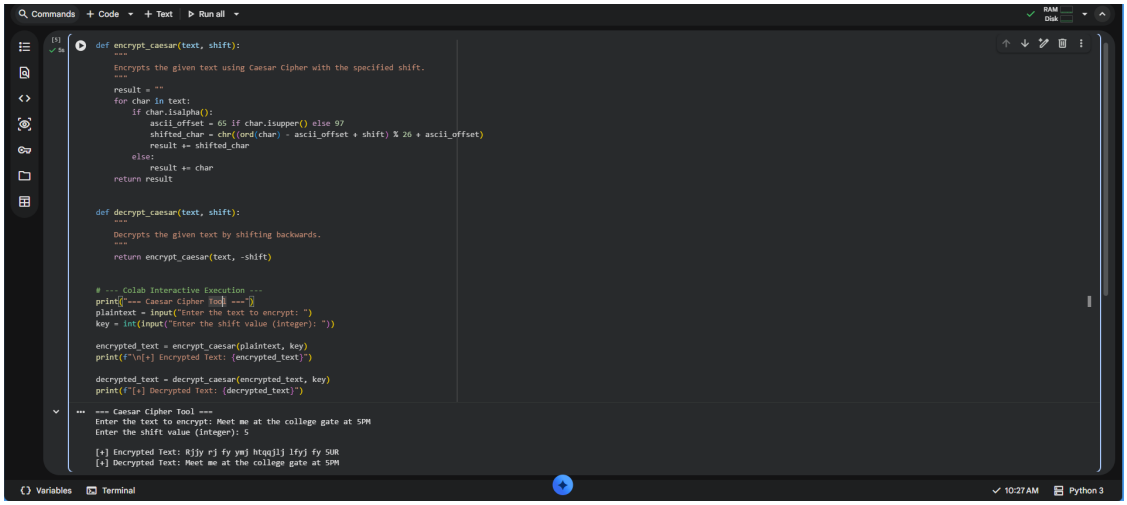
The program was executed with a real-life custom example, and the output was recorded below.

Custom Input Text (Real-life example): Meet me at the college gate at 5PM

Shift Key Used: 5

Observed Encrypted Output (Ciphertext): Rjjy rj fy ymj htqqjlj lfyj fy 5UR

Observed Decrypted Output (Plaintext): Meet me at the college gate at 5PM



Observation: Numeric characters (5PM) and whitespace remained unchanged in both the encrypted and decrypted outputs, confirming that the isalpha() check correctly isolates alphabetic characters for shifting while passing all other characters through unmodified.

8. Viva Questions

Q1. Real-Life Application: Why is the Caesar cipher no longer used for modern secure communications like WhatsApp, military operations, or Online Banking? Explain with respect to brute-force vulnerability.

Ans: The Caesar cipher has only 25 possible keys (shifts 1–25), which makes it trivial for an attacker to try every shift value and break the ciphertext in a fraction of a second — this is known as a brute-force attack. It is also vulnerable to frequency analysis, since the relative frequency pattern of letters in the plaintext language is preserved (just relabeled) in the ciphertext. Because of this negligible keyspace and lack of computational resistance, it offers no real security against automated attacks. Modern systems instead use algorithms such as AES or RSA, whose keyspaces (2^{128} or larger) are computationally infeasible to brute-force even with powerful modern computers.

Q2. Algorithmic Logic: In Python, why is it necessary to subtract and then add the ASCII offset (65 or 97) during the mathematical shift calculation?

Ans: Subtracting the offset converts the letter into its zero-based alphabetic position (A/a = 0, B/b = 1, ..., Z/z = 25), which is required because the shift-and-modulo (% 26) arithmetic is only valid over a 0–25 range. Working directly with raw ASCII values would make the modulo operation produce

incorrect results and could shift a character outside the alphabetic ASCII range entirely. After the shift and modulo are applied, the offset is added back to convert the resulting position back into a valid ASCII character in the correct case (uppercase or lowercase).

Q3. Code Efficiency: How did your Python script handle numbers and whitespace characters during the encryption loop?

Ans: The script uses the char.isalpha() check to test whether each character is a letter. If a character is not a letter — such as a digit, space, or punctuation mark the else branch executes, and the character is appended to the result unchanged. This means digits and whitespace pass through the cipher untouched, preserving the readability and structure of the message, while only alphabetic characters are shifted. This behaviour was verified in both Test Case 1 ('2026!' remained unchanged) and the custom observation ('5PM' and spaces remained unchanged).

9. Conclusion

The Caesar Cipher encryption and decryption program was successfully implemented in Python and executed in Google Colab. The manual calculation matched the program's output, and both the provided test cases as well as a custom real-life input were verified successfully. The experiment demonstrated the working of substitution ciphers and modular arithmetic, while also highlighting the cipher's vulnerability to brute-force and frequency analysis attacks, motivating the need for stronger modern cryptographic algorithms.